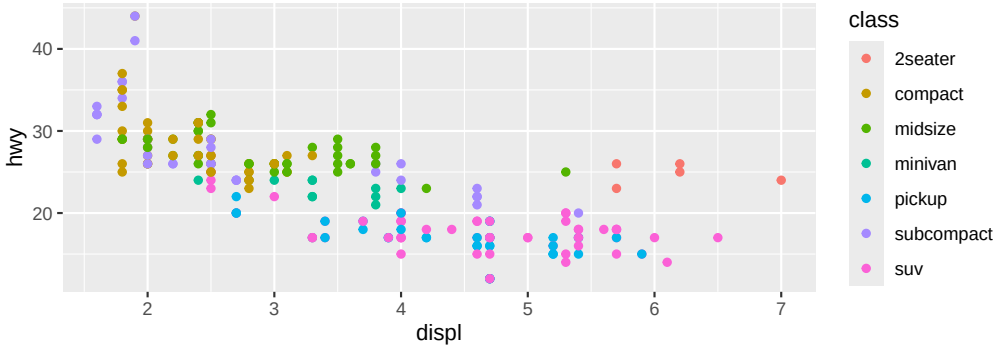
R4DS 05 – Layers

Aesthetic Mappings

Mapping vs. Setting Aesthetics

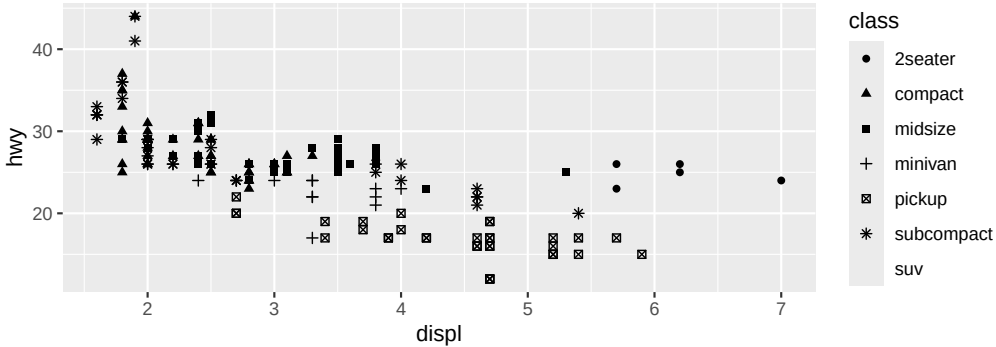
Map a variable to an aesthetic inside `aes()`:

```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point()
```



Shape Aesthetic

```
ggplot(mpg, aes(x = displ, y = hwy, shape = class)) +
  geom_point()
```



Only 6 shapes available by default — SUVs (7th group) go unplotted.

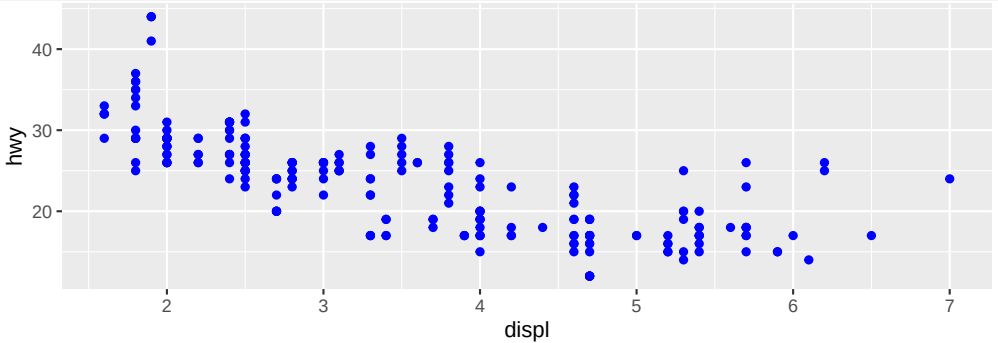
Size and Alpha

```
# Size: implies ranking --- not ideal for unordered categories
ggplot(mpg, aes(x = displ, y = hwy, size = class)) +
  geom_point()

# Alpha: same issue
ggplot(mpg, aes(x = displ, y = hwy, alpha = class)) +
  geom_point()
```

Mapping unordered categorical variables to ordered aesthetics (`size`, `alpha`) implies a ranking that doesn't exist.

Setting Aesthetics (Outside `aes()`)

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "blue")
```



Fixed values go **outside** `aes()`: `color = "blue"`, `size = 3`, `shape = 17`.

Exercises (Aesthetics)

1. Scatterplot of `hwy` vs. `displ` with pink filled-in triangles.
2. Why doesn't this produce blue points?

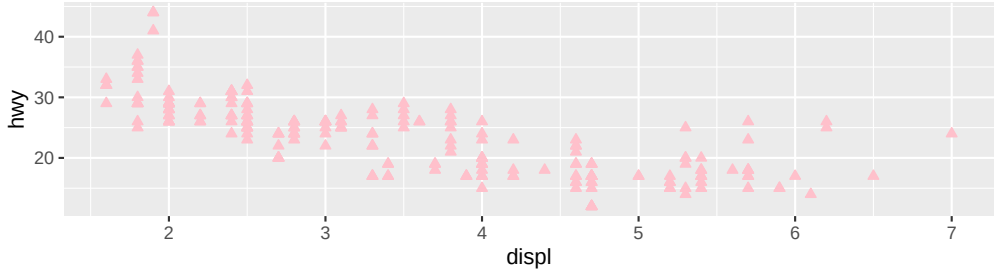
```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy, color = "blue"))
```

3. What does the `stroke` aesthetic do? Which shapes does it work with?
4. What happens with `aes(color = displ < 5)`?

Solutions (Aesthetics)

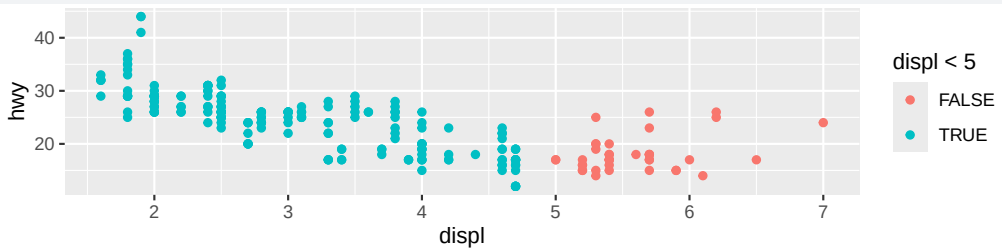
1.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "pink", shape = 24, fill = "pink")
```



2. `color = "blue"` is **inside** `aes()`, so ggplot maps the string `"blue"` as a categorical variable (red by default). Move it outside: `geom_point(color = "blue")`.
3. `stroke` controls border thickness for shapes 21–25 (filled shapes with separate border and fill).
4. Creates a logical variable (`TRUE`/`FALSE`), coloring points by whether `displ < 5`.

```
ggplot(mpg, aes(x = displ, y = hwy, color = displ < 5)) +
  geom_point()
```



Geometric Objects

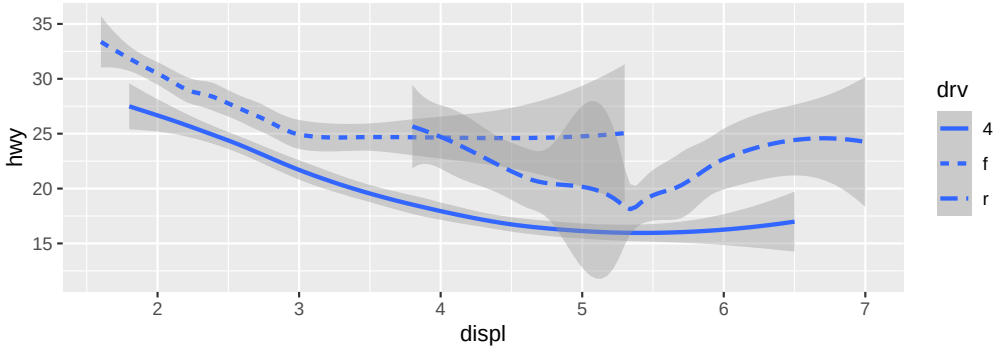
Different Geoms, Same Data

```
# Scatterplot
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()

# Smooth curve
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth()
```

Same data + different geom = different plot. Every geom has its own set of aesthetics.

Linetype Aesthetic

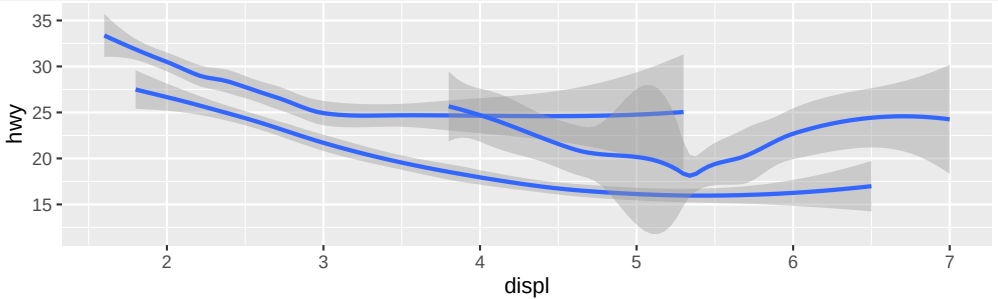
```
ggplot(mpg, aes(x = displ, y = hwy, linetype = drv)) +
  geom_smooth()
```



`geom_smooth()` uses `linetype` (not `shape`) to distinguish groups.

Grouping

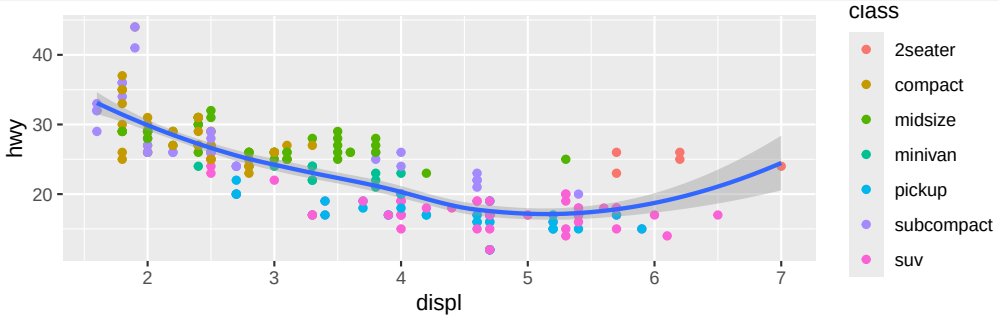
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(group = drv))
```



`group` splits data without adding a legend. Mapping to `color` or `linetype` adds a legend automatically.

Local vs. Global Aesthetics

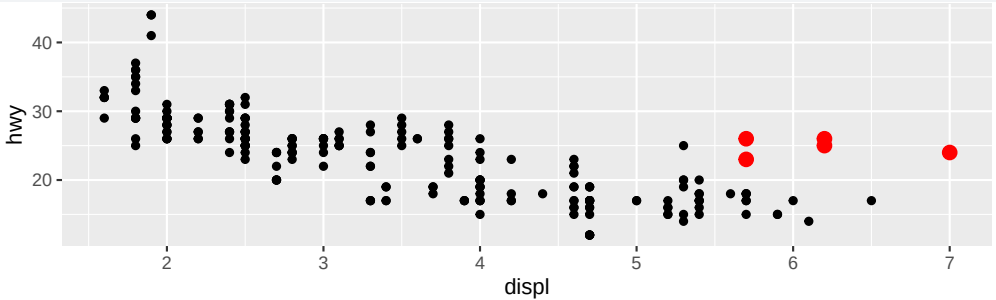
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth()
```



`color = class` in `geom_point()` only → one overall smooth line.

Different Data per Layer

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_point(
    data = mpg |> filter(class == "2seater"),
    color = "red", size = 3
  )
```



Override `data` in a layer to highlight a subset.

Geom Gallery

Geom	Plot Type	Key Aesthetic
`geom_point()`	Scatterplot	`shape`
`geom_smooth()`	Smooth curve	`linetype`
`geom_bar()`	Bar chart	`fill`
`geom_histogram()`	Histogram	`binwidth`
`geom_density()`	Density	`fill`, `alpha`
`geom_boxplot()`	Boxplot	`fill`
`geom_line()`	Line chart	`linetype`
`geom_jitter()`	Jittered scatter	`width`, `height`

40+ geoms available; extension packages add more.

Exercises (Geoms)

1. What geom for: line chart? boxplot? histogram? area chart?
2. What does `show.legend = FALSE` do? What if you remove it?
3. What does the `se` argument to `geom_smooth()` do?
4. Recreate these six plots (categorical variable is `drv`):

```
# (a) points + smooth
# (b) grouped smooth + points
# (c) color = drv on both geoms
# (d) color = drv on points only, one smooth
# (e) color = drv on points, linetype = drv on smooth
# (f) white border points (layer trick)
```

Solutions (Geoms) — 1–3

1. `geom_line()`, `geom_boxplot()`, `geom_histogram()`, `geom_area()`.
2. `show.legend = FALSE` hides the legend for that layer. Removing it shows the legend. Used to keep plots clean when legend is redundant.
3. `se = FALSE` removes the confidence interval ribbon around the smooth curve.

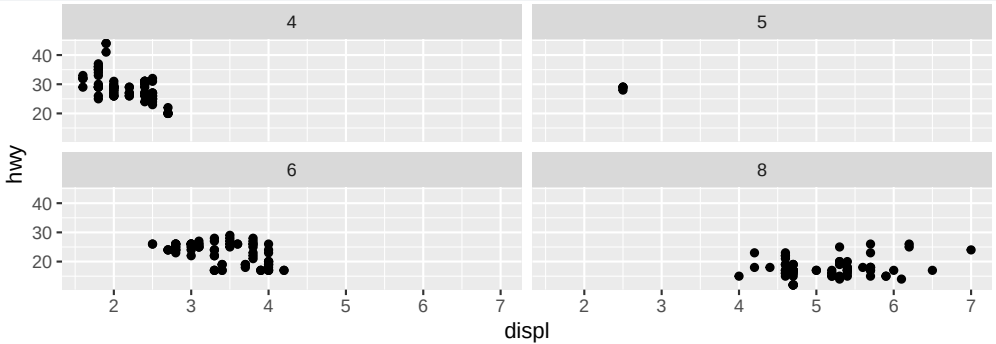
Solutions (Geoms) — 4

```
# (a)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() + geom_smooth(se = FALSE)
# (b)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(group = drv), se = FALSE) + geom_point()
# (c)
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() + geom_smooth(se = FALSE)
# (d)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) + geom_smooth(se = FALSE)
# (e)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(aes(linetype = drv), se = FALSE)
# (f)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(size = 4, color = "white") +
  geom_point(aes(color = drv))
```

Facets

facet_wrap() — One Variable

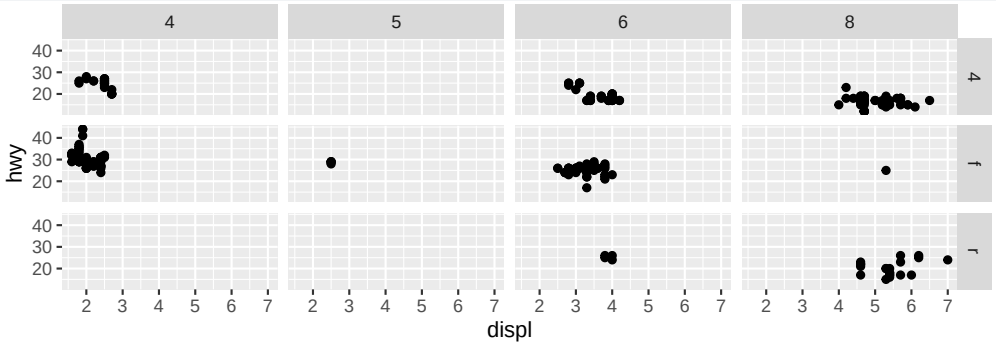
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~cyl)
```



Splits plot into subplots by one categorical variable.

facet_grid() — Two Variables

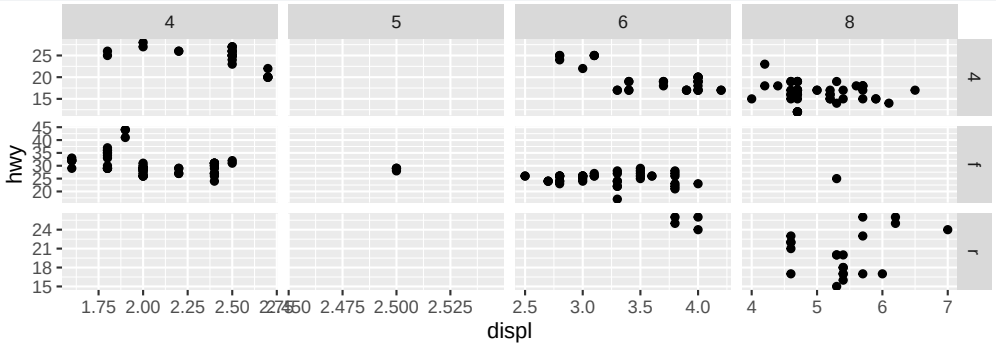
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_grid(drv ~ cyl)
```



`rows ~ cols`: drive train in rows, cylinders in columns.

Free Scales

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_grid(drv ~ cyl, scales = "free")
```



`scales = "free"` lets each panel have its own axis range. Also: `"free_x"`, `"free_y"`.

Exercises (Facets)

1. What happens if you facet on a continuous variable?
2. What do empty cells in `facet_grid(drv ~ cyl)` mean?
3. What does `.` do in `facet_grid(drv ~ .)` and `facet_grid(. ~ cyl)`?
4. Faceting vs. color aesthetic: advantages/disadvantages? Effect of dataset size?
5. Read `?facet_wrap`. What do `nrow`/`ncol` do? Why doesn't `facet_grid()` have them?
6. When placing a faceting variable across rows vs. columns, which makes comparison easier?
7. Recreate `facet_grid(drv ~ .)` using `facet_wrap()`. How do labels change?

Solutions (Facets) — 1–4

1. One facet per unique value — many panels, often not useful.
2. Empty cells = no observations for that combination (e.g., no 5-cyl front-wheel-drive cars).
3. `.` means "don't facet" on that dimension. `drv ~ .` = facet rows only; `. ~ cyl` = facet columns only.
4. Faceting avoids overplotting and is easier to read for many groups. Color is better for direct comparison (overlaid). With large data, faceting handles many groups better; color becomes cluttered.

5. `nrow`/`ncol` control the layout grid for `facet_wrap()`. `facet_grid()` doesn't need them — rows and columns are determined by the variables.
6. Facet rows share the x-axis $\rightarrow$ easier to compare horizontal distributions. Facet columns share the y-axis $\rightarrow$ easier to compare vertical distributions.
7.

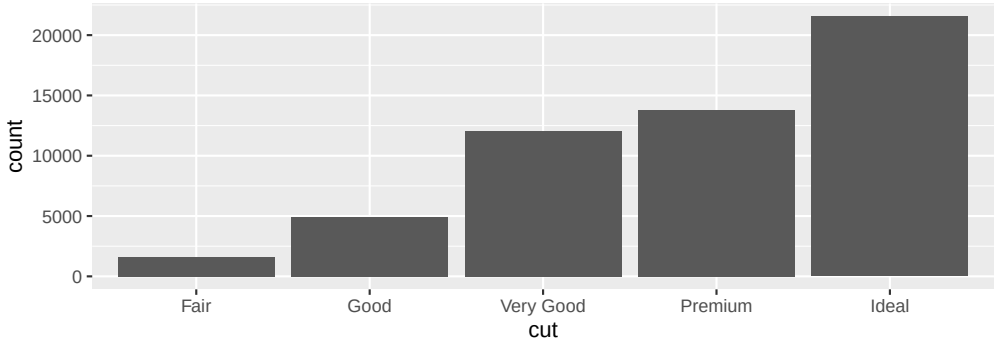
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~drv, ncol = 1)
```

Labels move from the right side (strip labels in `facet_grid`) to the top.

Statistical Transformations

Stats Behind Geoms

```
ggplot(diamonds, aes(x = cut)) +
  geom_bar()
```



The y-axis shows `count` — but `count` isn't in the data! `geom_bar()` uses `stat_count()` internally.

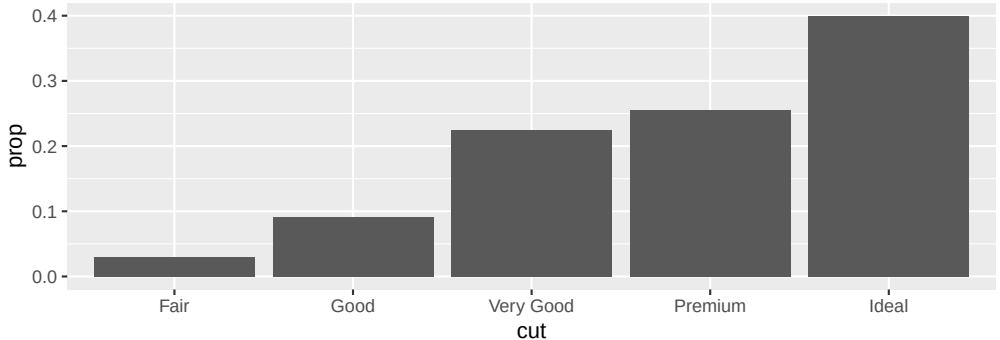
geom_bar() vs. geom_col()

```
# geom_bar(): stat = "count" (counts rows)
ggplot(diamonds, aes(x = cut)) +
  geom_bar()

# geom_col(): stat = "identity" (uses y values directly)
diamonds |>
  count(cut) |>
  ggplot(aes(x = cut, y = n)) +
  geom_col()
```

Use `geom_bar()` for counting; `geom_col()` when you already have summary values.

Proportion Bar Chart

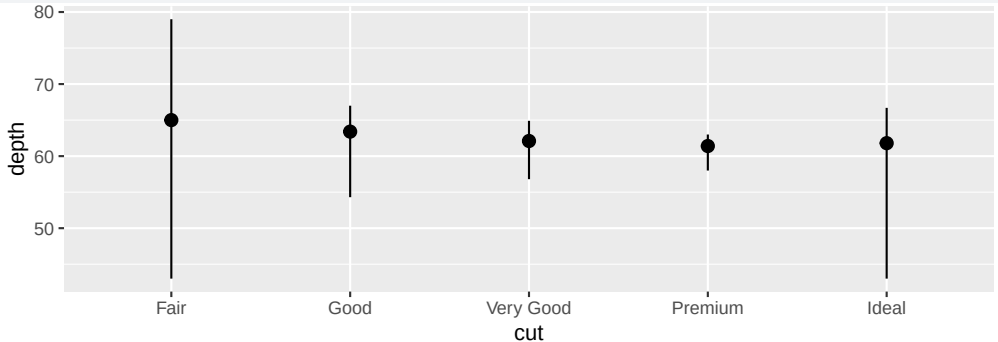
```
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = 1)) +
  geom_bar()
```



`after_stat(prop)` accesses computed proportions. `group = 1` forces proportions across the whole dataset.

stat_summary()

```
ggplot(diamonds) +
  stat_summary(
    aes(x = cut, y = depth),
    fun.min = min, fun.max = max, fun = median
  )
```



Use stats directly to emphasize the transformation. Default geom: `pointrange`.

Exercises (Stats)

1. Default geom for `stat_summary()`? Rewrite the previous plot using that geom.
2. What does `geom_col()` do? How is it different from `geom_bar()`?
3. List geom-stat pairs. What do they have in common?
4. What variables does `stat_smooth()` compute? What controls its behavior?
5. Why do we need `group = 1` in the proportion bar chart?

Solutions (Stats)

1. Default geom is `geom_pointrange()`. Rewrite:

```
ggplot(diamonds, aes(x = cut, y = depth)) +
  geom_pointrange(stat = "summary",
    fun.min = min, fun.max = max, fun = median)
```

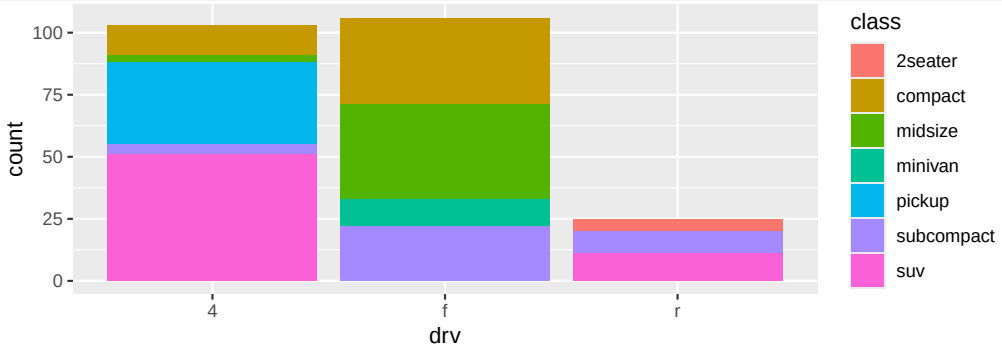
2. `geom_col()` uses `stat = "identity"` (bars from data values); `geom_bar()` uses `stat = "count"` (counts rows).
3. Common pairs: `geom_bar`/`stat_count`, `geom_histogram`/`stat_bin`, `geom_smooth`/`stat_smooth`, `geom_boxplot`/`stat_boxplot`. Each pair shares a default.

4. `stat_smooth()` computes `y` (predicted), `ymin`, `ymax`, `se`. Controlled by `method`, `formula`, `se`, `n`, `span`.
5. Without `group = 1`, proportions are computed **within each group** (each bar = 100%). With `group = 1`, proportions are relative to the total.

Position Adjustments

Bar Chart Fill

```
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar()
```



Mapping `fill` to a variable → stacked bars automatically.

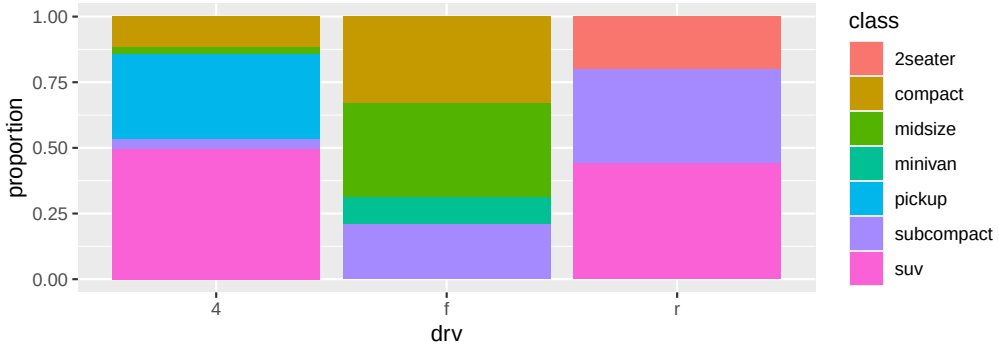
Position Options for Bars

```
# Stacked (default)
ggplot(mpg, aes(x = drv, fill = class)) + geom_bar()

# Proportional
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "fill")

# Side-by-side
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "dodge")
```

- `"fill"`: scales bars to 100% (compare proportions)
- `"dodge"`: places bars side by side (compare counts)

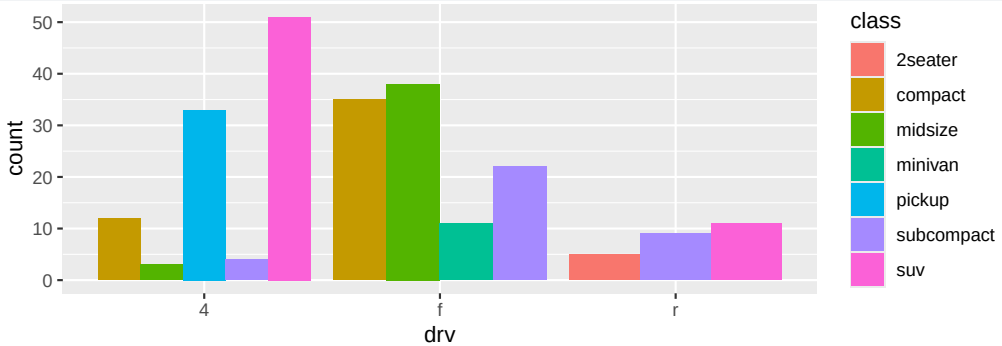
position = "fill" Example

```
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "fill") +
  labs(y = "proportion")
```



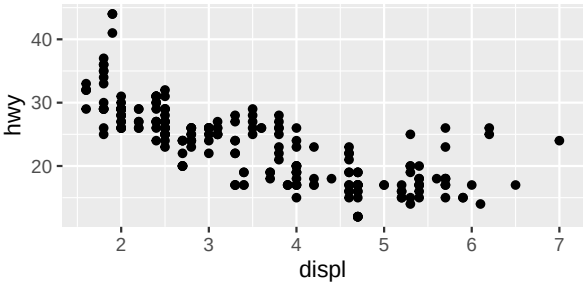
position = "dodge" Example

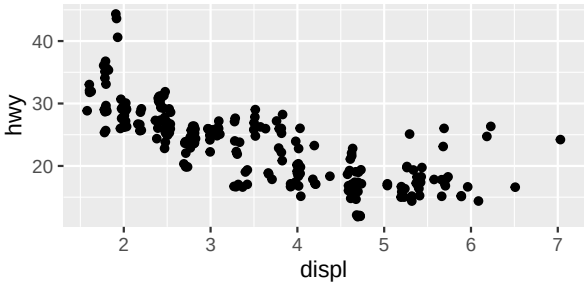
```
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "dodge")
```



Overplotting and Jitter

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_jitter()
```





`geom_jitter()` adds random noise to reveal overlapping points. Shorthand for `geom_point(position = "jitter")`.

Exercises (Position)

1. What's the problem with this plot? How to improve it?

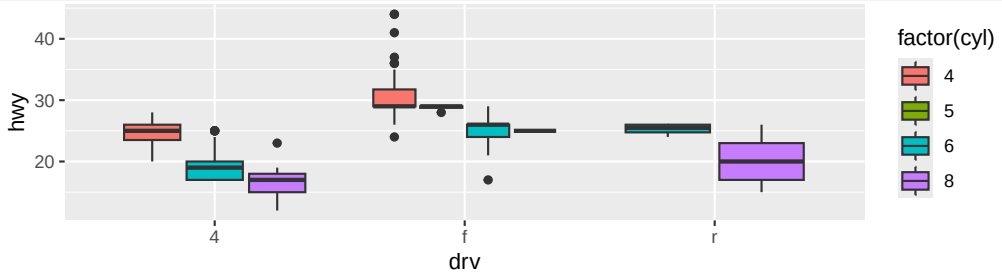
```
ggplot(mpg, aes(x = cty, y = hwy)) + geom_point()
```

2. Difference between `geom_point()` and `geom_point(position = "identity")`?
3. What parameters to `geom_jitter()` control the amount of jittering?
4. Compare `geom_jitter()` and `geom_count()`.
5. Default position for `geom_boxplot()`?

Solutions (Position)

1. Overplotting — many points overlap. Fix: use `geom_jitter()` or set `alpha`.
2. No difference — `"identity"` is the default position for `geom_point()`.
3. `width` and `height` control horizontal and vertical jitter amount.
4. `geom_jitter()` adds random noise; `geom_count()` maps point size to the number of overlapping observations. `geom_count()` preserves exact positions.
5. `position = "dodge2"`. Example:

```
ggplot(mpg, aes(x = drv, y = hwy, fill = factor(cyl))) +
  geom_boxplot()
```



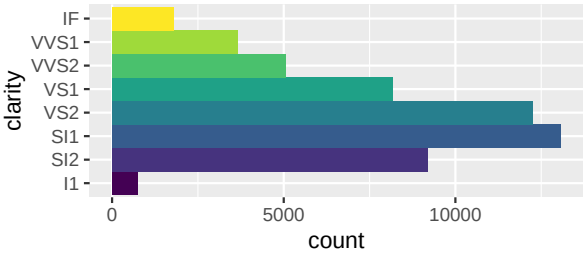
Coordinate Systems

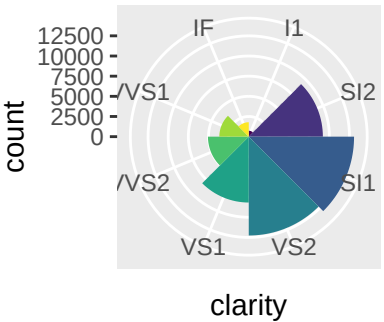
coord_flip() and coord_fixed()

```
# Flip axes (useful for long category names)
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot() + coord_flip()

# Fixed aspect ratio (1:1 scale)
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() + geom_abline() + coord_fixed()
```

`coord_polar()` — Polar Coordinates

```
bar <- ggplot(diamonds) +
  geom_bar(aes(x = clarity, fill = clarity),
           show.legend = FALSE, width = 1)

bar + coord_flip()
bar + coord_polar()
```





Polar coordinates turn a bar chart into a Coxcomb chart.

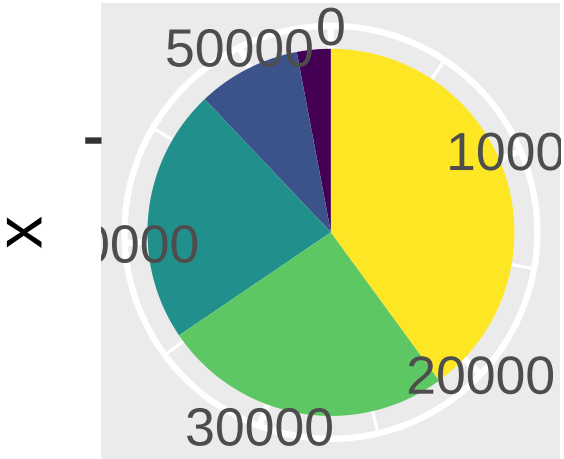
Exercises (Coordinates)

1. Turn a stacked bar chart into a pie chart using `coord_polar()`.
2. Difference between `coord_quickmap()` and `coord_map()`?
3. What does this plot tell you? Why is `coord_fixed()` important? What does `geom_abline()` do?

```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() + geom_abline() + coord_fixed()
```

1

```
ggplot(diamonds, aes(x = "", fill = cut)) +
  geom_bar(width = 1) + coord_polar(theta = "y") +
  theme(legend.key.size = unit(0.3, "cm"))
```



2. `coord_map()` uses Mercator (or other) projection (more accurate, slower). `coord_quickmap()` uses a fast approximation that works well for small areas.
3. `hwy` is always higher than `cty`. `coord_fixed()` ensures a 1:1 aspect ratio so the 45-degree reference line (`geom_abline()`) is visually meaningful.

The Layered Grammar

The Full ggplot2 Template

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
    mapping = aes(<MAPPINGS>),
    stat = <STAT>,
    position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

Seven parameters: data, mappings, geom, stat, position, coordinate system, facets.
In practice, ggplot2 provides sensible defaults for everything except data, mappings, and geom.